

COLECCIÓN RUST PÍXEL A PÍXEL

Cuaderno 1 de Rust

Descubriendo la programación y

Filtros digitales

Recomendado a partir de los 16 años años



Prototipo Editorial

Laboratorio de Código Instantáneo

Índice General

1. 🦀 Introducción	1
1.1. 📄 Descripción de los capítulos	1
1.2. Qué es un repositorio y qué es Git	1
1.2.1. Instalación de Git en Linux	2
1.2.2. Instalación de Git en MacOS	2
1.2.3. Instalación de Git en Windows	3
1.3. ⬇ Descargar los programas de ejemplo del cuaderno con Git	3
1.4. 📁 Estructura del Repositorio (curso-rust-institutos)	3
2. 🦀 Qué es un programa y por qué elegimos Rust	4
3. 🦀 Conceptos que necesitarás recordar	4
3.1. La información que maneja un programa	4
3.1.1. Tipos de datos reales	5
3.2. 🗣 El altavoz de tu código: Salida estándar con println!()	6
3.2.1. Los «Anclajes» u «Ojos de buey» {}	6
3.2.2. El limitador de decimales {:.N}	6
3.3. El proceso de compilación de un programa	7
3.4. Qué es una variable en programación	8
3.4.1. Variables inmutables	8
3.4.2. Variables mutables	8
3.4.3. Comentarios de línea	8
3.5. Divide y vencerás: ¿Qué es una función (fn)?	8
3.6. A los mandos de la Playground	8
3.6.1. El panel de control	8
3.6.2. El botón mágico: [SHARE]	8
3.6.3. Activando superpoderes: ¿Qué es un crate?	8
3.6.4. Lanzar el dado sin utilizar crates	8
3.6.5. Aprendiendo a leer el «idioma» del compilador	8
3.6.6. Tu copiloto digital: Cómo preguntar a la IA	8
3.6.7. 🛑 Las 3 reglas para no volverte dependiente de la IA	8
3.7. Las notas que el ordenador ignora: Los comentarios	8
3.8. Cuando los planes fallan: Errores de escritura vs. Errores de lógica	8
4. 🦀 Laboratorio	8
5. 🦀 Tipos de datos y operadores en Rust	8
6. 🦀 Subiendo de nivel: Instalación de un Entorno de Desarrollo Profesional	8
7. 🦀 Tratamiento de imágenes con Rust	8
8. 📄 Anexo	8

1. Introducción

¡Te damos la bienvenida al fascinante mundo de la programación!

Tienes en tus manos el primer cuaderno de una serie que iremos publicando paso a paso para acompañarte en esta aventura. No necesitas ninguna experiencia previa, ni saber de informática, ni ser un genio de las matemáticas. El único objetivo de este cuaderno es romper el miedo. Vamos a descubrir juntos cómo piensa un programador y cómo dar tus primeras órdenes a un ordenador. Para que no tengas que complicarte instalando programas raros ni configurando pantallas difíciles, trabajaremos directamente desde una página web en Internet. Usaremos la aplicación web Rust Playground como si fuera un lienzo en blanco para pintar nuestras ideas, crear historias y diseñar nuestros propios juegos. ¡Empezamos!

1.1. Descripción de los capítulos

Para que no te pierdas en esta aventura, hemos dividido este cuaderno en varias etapas diseñadas para ir ganando «superpoderes» como programador poco a poco:

- **El Despegue (Capítulos 1 al 3):** Aprenderemos cómo se comunica un programador con el ordenador, dónde guardamos la información y cómo usar la Playground de Rust desde el navegador sin instalar nada.
- **El Laboratorio de Pruebas (Capítulo 4):** Nos remangaremos para experimentar con código real, aprenderemos a tomar decisiones con condiciones y a repetir tareas usando bucles en proyectos divertidos.
- **Profundizando en Rust (Capítulo 5):** Viajaremos al interior de la memoria del ordenador para entender los tipos de datos y los operadores lógicos que hacen que la magia funcione.
- **El Entorno Profesional (Capítulos 6 y 7):** Dejaremos atrás el navegador de internet para instalar Rust y Visual Studio Code en tu ordenador como tienen en realidad los ingenieros de software. ¡Terminaremos el cuaderno creando programas capaces de leer y aplicar filtros visuales a fotos reales!

1.2. Qué es un repositorio y qué es Git

Cuando los programadores trabajan en un proyecto (como un videojuego o este curso de Rust), escriben decenas de archivos con código. Para que no se vuelva un caos de carpetas y archivos, con nombres como *proyecto_final_version3_definitiva_ESTA_SI.rs*, utilizamos una herramienta mágica llamada Git.

- **¿Qué es Git?:** Git es como una máquina del tiempo para tus archivos de código. Se encarga de vigilar tu proyecto y guardar «puntos de control» (como en los videojuegos). Si haces un cambio en tu código o el programa se rompe y no sabes qué ha pasado, puedes pulsar un botón y volver exactamente al momento en que todo funcionaba perfectamente. Sin embargo, nosotros no utilizaremos Git para realizar esta tarea, que se denomina control de versiones. Lo utilizaremos únicamente para descargarnos todo el material

asociado al cuaderno1 y el propio cuaderno1 desde Internet hasta una carpeta que elijamos en nuestro ordenador personal.

 Crea una carpeta denominada proyectos-rust en cualquier parte de tu disco duro, en donde almacenarás todos los archivos que vayamos escribiendo.

- **¿Qué es un Repositorio?:** Es el lugar donde se guardan todos los archivos de un proyecto junto con todo el historial de su «máquina del tiempo». Es decir, el repositorio guarda todas las versiones en el tiempo para cada archivo que almacena y su evolución desde que se creó el archivo en el repositorio hasta su estado actual en el mismo.
- **¿Qué es GitHub?:** Git funciona en tu propio ordenador, pero si quieres compartir tu caja fuerte digital de ficheros con el mundo, necesitas una nube. GitHub es como el «Instagram» o la red social de los programadores. Es una plataforma en Internet donde guardamos nuestros repositorios para que otras personas puedan ver nuestro código, descargarlo o colaborar con nosotros. Todo el material de este curso está guardado en un repositorio de GitHub llamado curso-rust-institutos. Luego explicaremos cómo puedes descargarlo a tu ordenador.

1.2.1. Instalación de Git en Linux

Si usas Linux, instalar Git es increíblemente rápido desde la terminal. Abre tu terminal favorita y escribe el comando según tu sistema:

- En **Ubuntu / Debian / Linux Mint:**

```
sudo apt update && sudo apt install git -y
```

- En **Fedora:**

```
sudo dnf install git -y
```

Para comprobar que se ha instalado bien, escribe el comando `git --version`. Si sale un número de versión, ¡ya lo tienes!

1.2.2. Instalación de Git en MacOS

En Mac, el propio sistema te lo pone muy fácil.

1. Abre la Terminal (búscala en el Launchpad o con Spotlight presionando Cmd + Espacio).
2. Escribe el siguiente comando y pulsa Enter:

```
git --version
```

1. Si no lo tienes instalado, aparecerá una ventana emergente de Apple preguntándote si deseas instalar las Herramientas de línea de comandos de Xcode. Haz clic en Instalar y acepta las condiciones. ¡El sistema hará todo el trabajo por ti!

Para comprobar que se ha instalado bien, escribe el comando `git --version`. Si sale un número de versión, ¡ya lo tienes!

1.2.3. Instalación de Git en Windows

Para poner Git en Windows, seguiremos el método oficial:

1. Entra en la página web oficial: `git-scm.com` y descarga el instalador para Windows.
2. Ejecuta el archivo descargado. Te saldrá un asistente con muchas opciones de configuración. No te compliques: haz clic en «Next» (Siguiente) a todo dejando las opciones por defecto hasta que termine la instalación.
3. Al finalizar, tendrás un nuevo programa instalado llamado Git Bash. Esta es una terminal especial para Windows desde la cual podrás usar todos los comandos de Git exactamente igual que si estuvieras en Linux o Mac.

Existen varias formas de abrir la terminal Git Bash en Windows pero unas de las más sencillas son las siguientes:

- **Desde el menú Inicio:** Haz clic en el botón de Inicio de Windows, busca la carpeta Git en la lista de aplicaciones y haz clic sobre el icono de Git Bash.
- **Desde el Explorador de archivos (En cualquier carpeta):** Haz clic derecho en una zona vacía de cualquier carpeta o en el Escritorio. En el menú contextual, selecciona Git Bash Here (o Mostrar más opciones > Git Bash Here en Windows 11). Esto abrirá la terminal directamente ubicada en esa ruta.

Para comprobar que se ha instalado bien, escribe el comando `git --version`. Si sale un número de versión, ¡ya lo tienes!

1.3. Descargar los programas de ejemplo del cuaderno con Git

Ahora que ya tienes la herramienta Git en tu ordenador, vas a utilizar tu primer comando de programación para traerte una copia exacta de toda nuestra caja fuerte del curso a tu equipo. A esto los programadores lo llamamos clonar un repositorio.

Abre tu terminal -en la carpeta `proyectos-rust` que hemos creado antes o en cualquier otra- (o Git Bash si estás en Windows) y escribe la siguiente orden mágica:

```
git clone https://github.com/vsorianano58/curso-rust-institutos.git
```

1.4. Estructura del Repositorio (curso-rust-institutos)

(Esto lo completaremos al final)

2. Qué es un programa y por qué elegimos Rust

Un programa informático es, en realidad, algo muy parecido a una receta de cocina. Es una lista ordenada de instrucciones detalladas que le damos a un ordenador para que realice una tarea concreta. Tu videojuego favorito, la aplicación de TikTok que usas en el móvil, el buscador de Google o el sistema de frenos de un coche moderno... todo eso son programas. Los ordenadores son máquinas increíblemente rápidas y potentes, pero tienen un pequeño defecto: no tienen imaginación ni iniciativa propia. Un ordenador no hace lo que tú quieres que haga; hace exactamente lo que tú le ordenas que haga en el programa. Si le das una orden equivocada obtendrás una respuesta que seguramente no esperabas.

Para escribir estas recetas de instrucciones existen cientos de «idiomas» diferentes que llamamos lenguajes de programación. En este curso hemos elegido uno llamado Rust, y lo hemos hecho por tres razones muy potentes:

- **Es un lenguaje del futuro (y del presente):** Las empresas tecnológicas más importantes del mundo (como Microsoft, Google o la NASA) están usándolo ahora mismo para construir sus sistemas más modernos y seguros. Aprender Rust hoy es aprender la tecnología que dominará los próximos 20 años.
- **Es súper seguro:** A diferencia de otros lenguajes antiguos, Rust tiene un «asistente integrado» (el compilador) que es extremadamente estricto. Si cometes un error al escribir, Rust te avisará amablemente antes de que el programa se rompa, explicándote qué has hecho mal y cómo solucionarlo. Es como tener un tutor personal a tu lado.
- **Es ultra rápido:** Con Rust se pueden crear programas que se ejecutan a la velocidad del rayo consumiendo poquísima memoria, lo que lo hace ideal desde para programar naves espaciales hasta para los videojuegos con los gráficos más exigentes.

A continuación, vamos a ver cómo se organiza la información que va a manejar nuestra primera receta de Rust.

3. Conceptos que necesitarás recordar

En este apartado vamos a incluir algunos conceptos sobre Rust de la máxima importancia. Evidentemente no están todos los que son pero si un pequeño conjunto que te resultará crucial para entender el resto del cuaderno y podrás volver aquí para revisarlos cada vez que lo necesites.

3.1. La información que maneja un programa

Los programas informáticos se caracterizan por manejar información. Esta información pueden obtenerla de muchas formas, por ejemplo: puede estar incrustada en el mismo programa o, facilitada por el usuario a través del ratón o el teclado, mediante una conexión a una cámara web, mediante la lectura de un código de barras, mediante la lectura de ficheros

que recibe desde Internet o desde otras vías; existen muchas otras formas mediante las cuales un programa puede recibir información.

En base a las fuentes de información citadas anteriormente, podríamos clasificar la información que puede recibir y por tanto manejar un programa en muchas categorías pero, vamos a simplificar este proceso y a distinguir solamente tres categorías de información:

- Numérica (representada por números enteros y decimales)
- Texto (representado por una agrupación de letras o caracteres)
- Booleanos, que solo tienen dos valores (verdadero y falso)

Lo primero a resaltar es que mientras en matemáticas el conjunto de los números es infinito, veremos luego que en informática no lo es y tiene valores máximos y mínimos que pueden alcanzar.

Un texto puede contener numerosos caracteres. Considérense los caracteres que contiene un libro cuyo contenido puede ser manejado íntegramente por un programa informático.

Sin embargo, el conjunto de los booleanos solo tiene dos valores, **true** (verdadero) y **false** (falso) pero, en informática son tan valiosos como los anteriores.

3.1.1. Tipos de datos reales

En el apartado anterior hablamos de forma muy general de **números**, **textos** y **booleanos**. Sin embargo, para programar de verdad en Rust, debemos entender que el ordenador solo entiende de interruptores eléctricos (**unos** y **ceros**).

Nota: Más adelante en el tema, en el apartado 5.1.- *Tipos de datos en Rust* desarrollaremos con mayor rigor el concepto que introducimos en este apartado. Lo hacemos así para que puedas entender los ejemplos de programas que vamos a estudiar sin adentrarnos todavía en una justificación teórica que es algo compleja y que dejamos para más tarde, cuando ya hayas utilizado estos conceptos desde la práctica con los ejemplos.

Para que esos unos y ceros se conviertan en una **edad**, en tu **nombre** o en el **precio** de un videojuego, Rust necesita saber el Tipo de Dato. Un tipo de dato es, simplemente, la plantilla que le dice al ordenador cómo traducir esos unos y ceros de la memoria RAM.

Aunque existen muchos tipos, por ahora solo necesitas dominar de momento estos cuatro tipos sagrados:

- **Enteros (i32):** Sirven para contar cosas completas que no se pueden partir. Tu edad, el año actual o el número de vidas de un personaje. Rust los llama **i32** (un entero de 32 bits). En la memoria ocupan un espacio de tamaño fijo.
- **Decimales (f64):** Sirven para medir cosas precisas. El precio de una golosina (1.50), la distancia en millas (5.34) o la temperatura (23.6). Rust los llama **f64** (número en coma flotante flotante de 64 bits). El ordenador los guarda usando una parte para el número y otra para saber dónde va el punto decimal. ¡Ojo! En programación usamos el punto ., nunca la coma , para los decimales.
- **Texto (&str y String):** Son cadenas de letras, números y símbolos unidos, siempre encerrados entre comillas (como «Halcón68»). En la memoria, el ordenador los guarda como una lista de caracteres seguidos.

- **Booleanos (bool)**: Es el tipo más sencillo del mundo. Solo tiene dos valores posibles: **true** (verdadero) o **false** (falso). En la memoria ocupan el mínimo espacio posible: un único interruptor encendido o apagado. Sirven para tomar decisiones (¿el juego ha terminado?, ¿el usuario es mayor de edad?).

3.2. 📢 El altavoz de tu código: Salida estándar con `println!()`

Un programa que calcula cosas en silencio dentro del ordenador pero no te las muestra es un programa inútil. Necesitamos un **altavoz** para comunicarnos con el usuario humano, y en Rust ese altavoz es la orden **`println!()`**.

La palabra viene del inglés Print Line (imprimir línea). Su funcionamiento parece sencillo, pero esconde dos herramientas de diseño avanzadas que verás constantemente en el laboratorio:

3.2.1. Los «Anclajes» u «Ojos de buey» `{}`

Cuando quieres mostrar texto combinado con variables, no puedes tirarlo todo junto. Rust utiliza los corchetes `{}` como marcadores de posición o **huecos para rellenar**. Piensa en ellos como un gancho donde vas a colgar tus datos en orden. Por ejemplo:

```
println!("Hola {}, tienes {} años.", nombre, edad);
```

Rust leerá esa línea y la imprimirá en la pantalla colocando el valor de nombre en el primer `{}` y el valor de edad en el segundo `{}`. Si cambias el orden de las variables al final, la frase cambiará por completo.

3.2.2. El limitador de decimales `{:.N}`

A los ordenadores se les da tan bien la precisión matemática que, si calculas una división o una conversión de kilómetros a millas, a veces te arrojan resultados como 8.59432210943. ¡Eso en una pantalla queda feísimo!

Para solucionarlo, Rust te permite tunear el marcador `{}` añadiendo instrucciones dentro. La más utilizada es `{:.2}`.

- Los dos puntos `:` en el interior significan: «Atención, voy a darte una instrucción de formato».
- El punto y el número dos `.2` significan: «Corta el número decimal y muestra solo dos dígitos después del punto».
- Se puede generalizar a N decimales donde N puede valer 2, 3, 4,... `{:.N}`.

Ejemplo:

Si el **resultado** real es 5.14367, la siguiente línea,

```
println!("Distancia: {:.3} km", resultado);
```

mostraría: Distancia: 5.143 km

3.3. El proceso de compilación de un programa

Un poco más adelante te explicaremos de forma más detallada el funcionamiento de la **Playground de Rust**. Es básicamente una aplicación web que te permite escribir en el lado izquierdo tu programa Rust y, si no tienes errores, cuando le das al botón **[RUN]** ves en la parte derecha el resultado de ejecutar el programa. Si cometes errores en la escritura de tu programa la Playground no generará ningún resultado. Te marcará los mensajes de error en rojo y te dará normalmente pistas para subsanarlos. Es posible que no sigas las recomendaciones de lo que Rust considera un estilo perfecto. En tal caso te entregará mensajes en color amarillo para avisarte pero también ejecutará el programa entregándote los resultados.

Vamos a utilizar por primera vez la Playground.

Pulsa sobre el enlace: [Rust Playground oficial](#)

3.4. Qué es una variable en programación

3.4.1. Variables inmutables

3.4.2. Variables mutables

3.4.3. Comentarios de línea

3.5. Divide y vencerás: ¿Qué es una función (fn)?

3.6. A los mandos de la Playground

3.6.1. El panel de control

3.6.2. El botón mágico: [SHARE]

3.6.3. Activando superpoderes: ¿Qué es un crate?

3.6.4. Lanzar el dado sin utilizar crates

3.6.5. Aprendiendo a leer el «idioma» del compilador

3.6.6. Tu copiloto digital: Cómo preguntar a la IA

3.6.7. 🚫 Las 3 reglas para no volverte dependiente de la IA

3.7. Las notas que el ordenador ignora: Los comentarios

3.8. Cuando los planes fallan: Errores de escritura vs. Errores de lógica

4. 🚀 Laboratorio

5. 🚀 Tipos de datos y operadores en Rust

6. 🚀 Subiendo de nivel: Instalación de un Entorno de Desarrollo Profesional

7. 🚀 Tratamiento de imágenes con Rust

8. 📄 Anexo

